

Proyecto DronSAR

De SITL a vuelo real

TELEM3, mavlink-router y separación SYSID/COMPID entre receptor.py y vuelo.py

Guía técnica de arquitectura e implementación

Pixhawk 6X · Raspberry Pi 5 · ArduPilot · pymavlink

Control de versiones

Autor	Fecha	Versión	Comentarios
Nerea Gorostidi García	25/08/2026	0.99	Borrador inicial

Índice

Control de versiones	2
Índice	3
Introducción	5
Objetivo del documento	5
Documentos relacionados.....	5
1. Objetivo de este documento.....	5
2. Qué conexiones usar en la Raspberry Pi	6
2.1 UART dedicado a TELEM3 (control y telemetría)	6
2.2 Alimentación de la Raspberry Pi.....	6
2.3 Conexión de datos hacia la nube (independiente del vuelo)	7
2.4 Qué hacer con el resto de puertos TELEM	7
2.5 Pinout exacto de TELEM3 y cómo cablearlo	8
2.6 El cable: el que trae el kit, y cómo hacer uno si falta	9
3. Verificación previa: confirmar que TELEM3 está activo, antes de instalar nada	10
3.1 Activar MAVLink en el SERIALx de TELEM3 (Mission Planner)	10
3.2 Guardar de verdad: Write Params y Refresh Params	11
3.3 Prueba de bajo nivel: bytes crudos con minicom, antes de MAVLink	11
3.4 Prueba con pymavlink (heartbeat real).....	11
3.5 Si minicom no muestra nada: diagnóstico por descarte	12
3.6 Scripts de prueba reutilizables	12
4. Instalación y configuración de mavlink-router en la Raspberry Pi 5.....	12
4.1 Prerrequisitos	13
4.2 Instalación (compilación desde fuente)	13
4.3 Fichero de configuración: /etc/mavlink-router/main.conf	14
4.4 Arrancar como servicio systemd	15
4.5 Verificación antes de conectar receptor.py y vuelo.py	15
5. receptor.py y vuelo.py: elegir el modo de conexión por parámetro	16
5.1 Diseño del .env unificado	16
5.2 Cambios en receptor.py	16
5.3 Cambios en vuelo.py	17
5.4 Cómo alternar entre SITL y real.....	17
6. SYSID / COMPID: por qué hay que añadirlo y cómo	18
6.1 Es necesario añadir identificación SYSID/COMPID propia a cada proceso	18
6.2 Convención propuesta	18
6.3 Implementación	19

7. Procedimiento de prueba y validación.....	19
7.1 Banco, sin hélices	19
7.2 Banco, sin hélices, con comando de armado	19
Qué revisar si el armado no se acepta	20
7.3 Con hélices, dron sujeto/atado	20
7.4 Vuelo corto supervisado.....	20
8. Resumen: .env de ejemplo completo.....	20
8.1 Checklist final	22
Apéndice C — Scripts de prueba de la conexión serie TELEM3	22
C.1 test-serial.py — loopback en la propia Raspberry Pi	22
C.2 test-mavlink.py — heartbeat real contra la Pixhawk.....	23
C.3 test-arm-serial.py — ciclo completo de armado y desarmado.....	23
Por serie directo, o a través de mavlink-router	24
C.4 test-estado.py — panel en vivo de batería, GPS y RC.....	30

Introducción

Este documento es la guía técnica detallada del paso de SITL a vuelo real con la Pixhawk 6X por TELEM3: cableado, pinout, instalación de mavlink-router, cambios en receptor.py y vuelo.py, identificación SYSID/COMPID, y los cuatro scripts de prueba de la conexión serie contruidos y validados durante el proyecto.

Objetivo del documento

Servir de referencia paso a paso, reproducible, para volver a montar o depurar la conexión TELEM3 en el futuro — incluyendo los problemas reales encontrados durante la instalación (el dispositivo `/dev/ttyAMA0` en vez de `/dev/serial0`, la calibración de radio, la compilación de mavlink-router) y cómo se diagnosticaron y resolvieron, no solo el procedimiento en el caso ideal.

Documentos relacionados

El recorrido completo del proyecto — desde el estudio del arte hasta el panel de control en la nube, en ocho etapas — está descrito en el siguiente documento, cuyos Apéndices B y C resumen el contenido de este dentro de ese recorrido más amplio:

Documento	Relación / comentarios
Arquitectura_Teleoperacion_Simulacion_Drones.pdf	Mapa completo del proyecto en ocho etapas y cómo encaja esta pieza dentro de él; consulta este documento para el detalle técnico paso a paso.
MAVLink_Explicado.pdf	Ampliación a nivel de sockets y configuración de la conexión MAVLink en general (Fase 2, modo SITL) — este documento cubre específicamente el paso a hardware real (Fase 5).

El código completo de receptor.py, vuelo.py y los cuatro scripts de prueba de este documento está publicado en <https://github.com/nereagorostidi/drone-edge-companion>. El código incluido aquí refleja el estado del proyecto en la fecha de este documento; el repositorio es la fuente actualizada.

1. Objetivo de este documento

El código actual de receptor.py y vuelo.py solo se ha probado en modo SITL: se conectan por red (UDP) contra un dron simulado que corre dentro de Mission Planner, en el mismo PC o en la misma red local. Ese modo es cómodo para desarrollar, pero no representa el caso real: en vuelo, no hay ningún simulador ni ninguna conexión de red hacia el autopiloto — hay que enviar los comandos MAVLink directamente al Pixhawk 6X físico, a través de un puerto serie (TELEM3), desde la propia Raspberry Pi que va a bordo del dron.

El objetivo de este documento es precisamente ese cambio: que receptor.py y vuelo.py dejen de depender exclusivamente del modo SITL/red y sean capaces también de hablar MAVLink por el puerto serie contra el Pixhawk real, sin dejar de poder usarse en SITL cuando convenga para seguir desarrollando. Cubre cuatro bloques:

- Qué conexiones físicas usar en la Raspberry Pi y por qué (radio de telemetría en TELEM1 para Mission Planner, UART directo en TELEM3 para la Pi).

- Instalación y configuración de mavlink-router en la Raspberry Pi 5, para que el puerto serie TELEM3 pueda ser compartido de forma segura por receptor.py y vuelo.py.
- Los cambios de código necesarios en receptor.py y vuelo.py para que, mediante un parámetro de entorno, se conecten al SITL (como hasta ahora) o al Pixhawk real a través de mavlink-router, sin duplicar lógica.
- El estado actual — y la implementación necesaria — de la identificación SYSID/COMPID de cada proceso, que hoy no está diferenciada y debería estarlo antes de volar con hardware real.

Contexto previo: Este documento asume que ya se ha completado la configuración previa de ArduPilot para TELEM3 (SERIALx_PROTOCOL=2, SERIALx_BAUD) y la conexión de Mission Planner por radio de telemetría en TELEM1, tal como se acordó previamente. Aquí nos centramos en el lado Raspberry Pi y en el código Python.

2. Qué conexiones usar en la Raspberry Pi

Antes de tocar software, conviene fijar qué lleva cada cable. La Raspberry Pi 5 del payload debe tener, en producción, exactamente estas conexiones relacionadas con el vuelo:

2.1 UART dedicado a TELEM3 (control y telemetría)

- TELEM3 del Pixhawk 6X entrega UART a nivel TTL de 3.3V (TX, RX, GND, y opcionalmente RTS/CTS). El GPIO UART de la Raspberry Pi 5 también es 3.3V, así que la conexión es directa: TX del Pixhawk → RX de la Pi, RX del Pixhawk → TX de la Pi, GND común. No se necesita ningún adaptador de niveles.
- No usar USB para este enlace: el USB del Pixhawk se reserva para configuración en banco (Mission Planner/QGroundControl conectado directamente), no para el enlace en vuelo con la Pi.
- En la Raspberry Pi 5, habilitar el UART por hardware con raspi-config (Interface Options → Serial Port), respondiendo "No" a la consola serie y "Sí" al hardware serial habilitado.
- Como la Pi 5 asigna por defecto el UART completo (PL011) a Bluetooth, si se necesita el enlace más fiable a baudios altos conviene liberar ese UART desactivando Bluetooth en /boot/firmware/config.txt con dtoverlay=disable-bt.

Importante — usar /dev/ttyAMA0, no /dev/serial0: En esta Raspberry Pi 5 concreta, el symlink /dev/serial0 NO apunta al UART físicamente cableado a TELEM3: apunta a ttyAMA10, un UART distinto que nunca recibe nada por esos pines. El UART que sí está cableado a GPIO14/GPIO15 (pines físicos 8 y 10, TELEM3) es /dev/ttyAMA0, confirmado por prueba directa (ver 3.3). Por eso, en este documento y en toda la configuración de este proyecto, se usa /dev/ttyAMA0 en vez de /dev/serial0. Si en el futuro se cambia de placa o de imagen del sistema operativo, no dar esto por sentado: repetir la verificación de 3.3 antes de asumir que /dev/serial0 es el correcto.

2.2 Alimentación de la Raspberry Pi

- En producción, la Raspberry Pi 5 se alimenta desde el UBEC 5V/8A conectado directamente a la salida de la batería LiPo 4S, en paralelo con la alimentación del Pixhawk (vía PM02) — nunca desde el propio Pixhawk ni desde los rieles de servo, que no tienen margen para la Pi 5 + HAT Hailo + módem + cámara.
- Para las pruebas de banco de este documento, si todavía no se dispone del UBEC, la Raspberry Pi puede alimentarse directamente de la toma eléctrica con su adaptador de corriente habitual, sin

pasar por la batería del dron. Esto es válido mientras el dron esté en banco y no se vaya a volar — en cuanto se monte para volar, la alimentación debe pasar a ser la del UBEC desde la LiPo, para no depender de un cable a la red eléctrica.

2.3 Conexión de datos hacia la nube (independiente del vuelo)

- El módem USB Huawei mantiene la conectividad 4G hacia el broker MQTT / EC2, y es independiente del enlace MAVLink; no compite por el puerto serie.

2.4 Qué hacer con el resto de puertos TELEM

- TELEM1 queda reservado en exclusiva para la radio de telemetría hacia Mission Planner, como ya se decidió.
- TELEM3 es el que se cablea a la Raspberry Pi, tal y como se describe en el punto 2.1.
- TELEM2 se deja libre, sin usar por ahora: es el puerto natural para añadir el día de mañana otro consumidor MAVLink (por ejemplo, un segundo enlace de radio, un módulo de telemetría de largo alcance, u otro companion computer) sin tener que reconfigurar nada de lo ya montado.
- El puerto USB del Pixhawk queda libre para depuración en banco (configuración con Mission Planner/QGroundControl conectado directamente), no para producción.

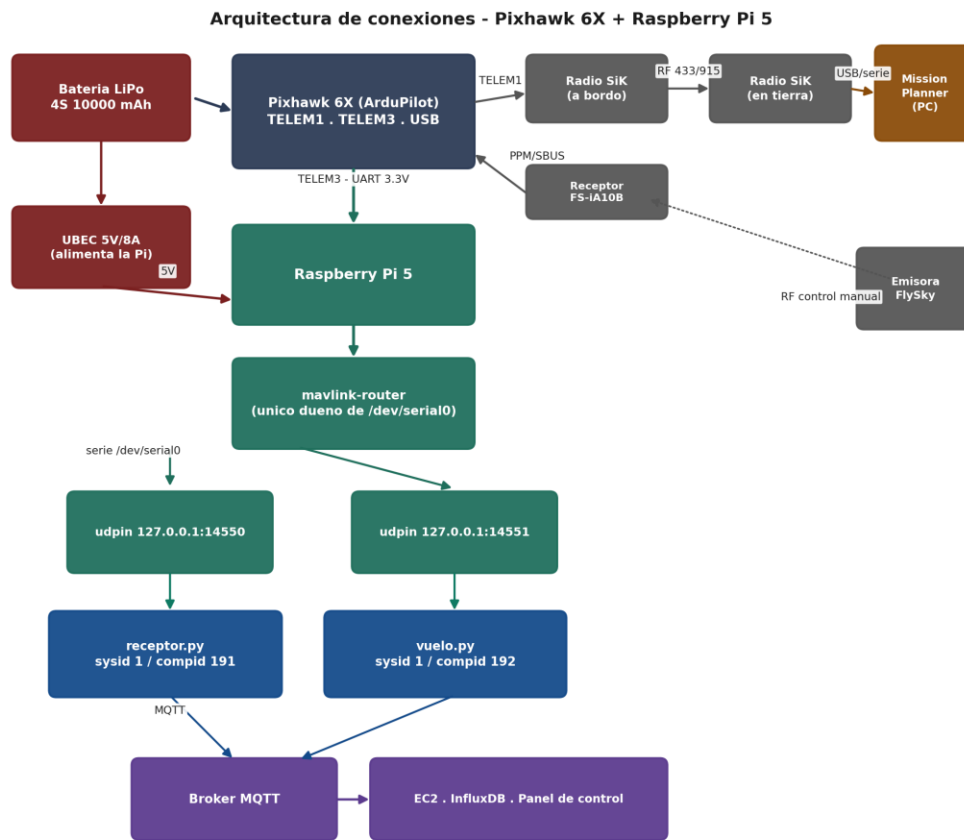


Figura 1. Arquitectura de conexiones física completa: alimentación, TELEM1 (radio, Mission Planner), TELEM3 (Raspberry Pi) y salida hacia la nube.

2.5 Pinout exacto de TELEM3 y cómo cablearlo

TELEM3 es un conector JST-GH de 1.25mm de 6 pines, y sigue el mismo pinout estándar que TELEM1/TELEM2 en toda la familia Pixhawk (confirmado en la documentación oficial de Holybro para el Pixhawk 6X). El orden de los pines, contando desde el pin 1, es siempre el mismo — lo único que cambia entre puertos es a qué UART interno del microcontrolador está conectado cada uno:

Pin	Señal	Qué hacer
1	VCC (+5V)	No conectar a la Raspberry Pi — la Pi tiene su propia alimentación (UBEC/adaptador), y unir esta línea a un GPIO la dañaría.
2	TX (sale del FC)	A RX de la Raspberry Pi — pin físico 10 (GPIO15).
3	RX (entra al FC)	A TX de la Raspberry Pi — pin físico 8 (GPIO14).

Pin	Señal	Qué hacer
4	CTS (entra al FC)	No conectar — solo hace falta si se activa control de flujo por hardware en SERIALx_OPTIONS, que dejamos desactivado (valor 0).
5	RTS (sale del FC)	No conectar, por el mismo motivo que CTS.
6	GND	A cualquier pin GND libre de la Raspberry Pi (por ejemplo pin 6, 9 o 14). Es el pin que más fácil se olvida, y sin él no hay comunicación posible aunque TX/RX estén bien — los niveles de la señal no tienen referencia común.

Resumen: Solo se conectan 3 hilos en total: TX, RX y GND. VCC, CTS y RTS se dejan sin conectar a nada.

2.6 El cable: el que trae el kit, y cómo hacer uno si falta

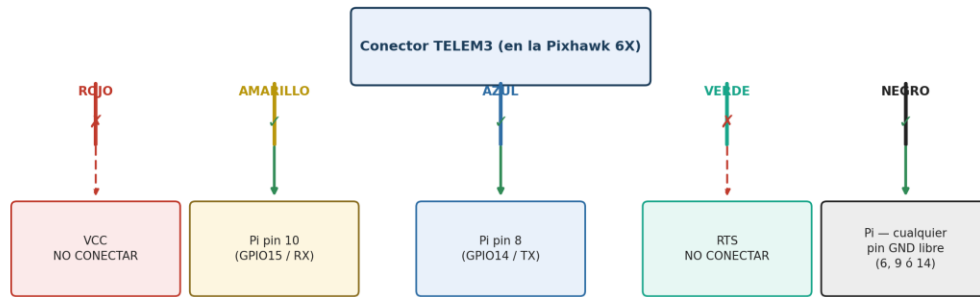
El kit de compra de la Fase 5 ya incluye un cable pensado exactamente para esto: un pigtail con conector JST-GH 1.25mm de 6 vías en un extremo (para la Pixhawk) y pines Dupont sueltos en el otro (para el header GPIO de la Raspberry Pi) — el mismo tipo de cable listado en la lista de compra original de este proyecto. No hace falta pelar ni crimpar nada: ambos extremos ya vienen terminados de fábrica.

- Es habitual que este tipo de cables, pensados para companion computers, no traigan el hilo de VCC (pin 1) de fábrica — es una medida de seguridad del propio fabricante, para que no sea posible alimentar accidentalmente la Raspberry Pi (u otra placa) desde la Pixhawk. Si tu cable solo trae 5 hilos en vez de 6, lo más probable es que falte precisamente ese, y no supone ningún problema: no lo necesitas.
- El color de cada hilo NO está estandarizado entre fabricantes — no te fíes de que "amarillo es siempre TX" en cualquier cable que compres. Lo único razonablemente universal es que el rojo suele ser VCC y el negro GND, por convención casi general. Para TX/RX, lo fiable es la posición dentro del conector (pines 2 y 3), no el color.
- Si tienes multímetro: identifica VCC por voltaje (~5V) y GND por continuidad contra un GND ya conocido de la placa (por ejemplo el pin 6 de TELEM2, que es GND en todos los TELEM); a partir de ahí, cuenta posiciones.
- Si no tienes multímetro: desenchufa el conector de la Pixhawk (es a presión, no está soldado) y mira el zócalo vacío en la placa — casi siempre hay un pequeño "1" serigrafiado junto al pin 1, o el primer pad tiene forma distinta (cuadrado) al resto (redondos). Desde ahí, cuenta 2=TX, 3=RX, 4=CTS, 5=RTS, 6=GND.

TELEM3 → Raspberry Pi 5

Esquema de conexión CORREGIDO

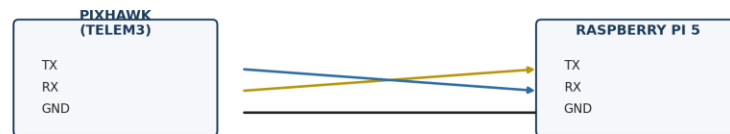
(sustituye al esquema en papel anterior)



En total: solo 3 cables conectados (amarillo, azul, negro).

Rojo y verde se dejan sueltos, sin conectar a nada.

¿Por qué se cruzan TX y RX? (recordatorio)



El TX de un lado siempre va al RX del otro (cruzados), y GND va directo con GND.

Figura 2. Ejemplo real de asignación de colores en un cable de este proyecto, una vez identificados por posición — sirve como referencia de formato, no asumas que tu cable use los mismos colores.

Si no dispones de ningún cable para TELEM3 y hay que hacerse uno desde cero, lo más práctico es comprar un pigtail JST-GH 1.25mm de 6 vías precrimpado a Dupont (el mismo tipo de producto referenciado en la lista de compra de este proyecto), en vez de crimpar a mano: el paso de 1.25mm es muy pequeño y requiere una crimpadora específica poco habitual fuera de un taller de electrónica; para un solo cable, sale más a cuenta y es más fiable comprarlo ya hecho.

3. Verificación previa: confirmar que TELEM3 está activo, antes de instalar nada

Antes de instalar mavlink-router tiene sentido comprobar, con la conexión más simple posible (un solo programa, sin repartir el puerto entre nadie), que el cableado y la configuración de ArduPilot son correctos. Si algo falla en este punto, es mucho más fácil de diagnosticar sin la capa añadida de mavlink-router — y de hecho, cualquier fallo aquí impediría igualmente que mavlink-router funcionase después.

3.1 Activar MAVLink en el SERIALx de TELEM3 (Mission Planner)

Con la Pixhawk 6X conectada por USB a Mission Planner, en Config → Full Parameter List: TELEM3 corresponde a SERIAL5 en el Pixhawk 6X (confirmado en la documentación oficial de ArduPilot/Holybro para esta placa). Configura:

- SERIAL5_PROTOCOL = 2 (MAVLink2).
- SERIAL5_BAUD = 57 (57600 baudios) — debe coincidir exactamente con lo que uses luego en minicom y en pymavlink.

Importante: El número de SERIALx que corresponde a TELEM3 puede variar entre modelos y revisiones de placa Pixhawk. Este documento asume un Holybro Pixhawk 6X, donde TELEM3 = SERIAL5; si en algún momento se cambia de modelo de autopiloto, hay que volver a verificar esta correspondencia en la documentación del fabricante antes de tocar parámetros.

3.2 Guardar de verdad: Write Params y Refresh Params

Un error muy fácil de cometer en Mission Planner: seleccionar el nuevo valor en el desplegable de un parámetro NO lo envía a la Pixhawk por sí solo — solo lo deja marcado como "modificado" en la pantalla (resaltado en azul). Hasta que no se pulsa Write Params, ese cambio vive solo en Mission Planner, y se pierde si se refresca la pantalla o se reconecta.

- Tras cambiar SERIAL5_PROTOCOL y SERIAL5_BAUD, pulsar Write Params.
- Pulsar Refresh Params, que relee todos los parámetros directamente desde la Pixhawk por MAVLink, no desde la caché de Mission Planner.
- Confirmar que el valor mostrado tras el refresh es el correcto y que ya NO aparece resaltado en azul (el resaltado indica un cambio pendiente de escribir, no un valor ya grabado).
- Los cambios de SERIALx_PROTOCOL requieren, además, un reinicio de la Pixhawk (desconectar/reconectar alimentación) para tomar efecto — un simple Write Params no basta.

3.3 Prueba de bajo nivel: bytes crudos con minicom, antes de MAVLink

Antes de probar con pymavlink, conviene confirmar que hay señal eléctrica llegando por el cable — sin depender todavía de que el protocolo MAVLink esté bien formado. Es la forma más rápida de distinguir un problema de cableado/baudrate de un problema de protocolo:

```
sudo apt install -y minicom
minicom -D /dev/ttyAMA0 -b 57600
```

- Flujo denso y continuo de caracteres binarios (ilegibles, no tienen que tener sentido): cableado y baudrate correctos. Se puede pasar a la prueba con pymavlink.
- Pantalla completamente vacía: hay un problema de cableado, de puerto físico equivocado, o el protocolo de SERIAL5 no está realmente activado (ver 3.5).

3.4 Prueba con pymavlink (heartbeat real)

```
python3 -c "
from pymavlink import mavutil
m = mavutil.mavlink_connection('/dev/ttyAMA0', baud=57600)
m.wait_heartbeat()
print('OK, sistema', m.target_system, 'componente', m.target_component)
"
```

Si aparece el mensaje OK, la conexión TELEM3 funciona de extremo a extremo — solo entonces tiene sentido pasar a instalar mavlink-router (sección 4).

3.5 Si minicom no muestra nada: diagnóstico por descarte

- Confirmar (de nuevo, tras un Refresh Params) que SERIAL5_PROTOCOL sigue en 2 y SERIAL5_BAUD en 57 — es fácil que un cambio no llegara a escribirse de verdad (ver 3.2).
- Confirmar que el cable está físicamente en el conector TELEM3, y no en TELEM1 o TELEM2 por error.
- Confirmar que el hilo de GND está realmente conectado — es, con diferencia, el fallo más fácil de pasar por alto, y por sí solo puede dejar la pantalla de minicom completamente vacía.
- Probar a intercambiar los hilos de TX y RX entre sí: es un error común y no conlleva ningún riesgo probarlo, ya que ambas señales son de 3.3V lógicos.
- Descartar la Raspberry Pi por completo con una prueba de loopback: desconectar el cable de la Pixhawk, puentear directamente TX con RX en los propios pines de la Raspberry Pi, y comprobar en minicom que lo que se teclea se ve reflejado en pantalla (eco). Si el eco funciona, el UART y minicom de la Pi están bien, y el problema está 100% en el cable o en la Pixhawk.
- Comprobar que ningún otro proceso tiene ya abierto el puerto: si mavlink-router está corriendo como servicio (sección 4.4), se queda con el dispositivo para él solo y minicom/pymavlink no verán nada aunque el cableado esté perfecto. Pararlo antes de cualquier prueba manual: `sudo systemctl stop mavlink-router.service` — y volver a arrancarlo después con `sudo systemctl start mavlink-router.service`.
- Si con todo lo anterior descartado el minicom sigue sin mostrar nada, no dar por sentado que `/dev/serial0` es el dispositivo correcto: verificarlo con un script directo (sección 3.6) contra `/dev/ttyAMA0`, que es el que resultó ser el correcto en esta Raspberry Pi 5 concreta.

3.6 Scripts de prueba reutilizables

Durante el diagnóstico de este proyecto se construyeron cuatro scripts pequeños, independientes de `receptor.py` y `vuelo.py`, pensados para aislar el enlace TELEM3 paso a paso sin depender de todo el sistema en marcha. El más simple (`test-serial.py`) fue el que resolvió el caso real de este proyecto: confirmó que el UART de la Raspberry Pi funcionaba perfectamente por `/dev/ttyAMA0`, mientras que `/dev/serial0` apuntaba a un UART distinto (`ttyAMA10`) que nunca estuvo cableado a TELEM3 — ver la nota de la sección 2.1. El Apéndice C recoge los cuatro completos, con su código y para qué sirve cada uno.

Recomendación: Guárdalos tal cual en la Raspberry Pi (por ejemplo en `~/drone-edge-companion/`) — son la forma más rápida de volver a comprobar el enlace TELEM3 en el futuro, sin tener que levantar mavlink-router ni los servicios completos.

4. Instalación y configuración de mavlink-router en la Raspberry Pi 5

Al pasar a hardware real aparece un problema que en SITL no existía: `receptor.py` necesita enviar comandos (armar, desarmar) y `vuelo.py` necesita leer telemetría a alta frecuencia, y ambos tienen que hacerlo contra el mismo Pixhawk al mismo tiempo. En SITL esto no era un problema porque Mission Planner reenviaba por red a cuantos puertos UDP hicieran falta, uno por proceso. Pero un puerto serie es un recurso exclusivo del

sistema operativo: dos procesos no pueden abrir `/dev/ttyAMA0` a la vez sin pisarse los datos que llegan y salen por TELEM3.

La solución es poner delante del puerto serie algo que haga de router: un único proceso que sea el que realmente abre `/dev/ttyAMA0`, y que reparta ese tráfico MAVLink hacia tantos consumidores como haga falta — en este caso, `receptor.py` y `vuelo.py`, cada uno por su propio endpoint de red local. Eso es exactamente lo que hace `mavlink-router`.

Es el mismo papel que cumplía MavProxy en el flujo de trabajo que ya conocemos con el SITL: MavProxy se ponía en medio del dron simulado (que en SITL habla por red, no por serie) y de Mission Planner, para que el simulador pudiera enviar su telemetría a la vez al GCS y recibir de vuelta las peticiones/comandos MAVLink de ese mismo GCS, sin que ambos extremos tuvieran que hablar directamente entre sí. `mavlink-router` hace lo mismo, pero con una diferencia de arquitectura importante: en vuelo real, Mission Planner ya no se conecta contra ese mismo punto — se conecta por su radio de telemetría en TELEM1, como se explicó en la sección 2. Es decir, hay dos "routers" distintos conviviendo, cada uno en su sitio: `mavlink-router` en la Raspberry Pi, exclusivamente para repartir TELEM3 entre `receptor.py` y `vuelo.py`; y el radio de TELEM1, como enlace aparte e independiente, para Mission Planner.

4.1 Prerrequisitos

- UART de TELEM3 habilitado y accesible como `/dev/ttyAMA0` (sección 2.1 ya completada; ojo, NO `/dev/serial0` en esta placa).
- Parámetros ArduPilot ya configurados en el SERIALx correspondiente a TELEM3: SERIALx_PROTOCOL=2 (MAVLink2) y SERIALx_BAUD acorde (57 para 57600 baudios, valor recomendado para un cable corto sin necesidad de Bluetooth desactivado; 921 para 921600 si se liberó el UART completo).
- Paquetes de compilación: `git`, `meson` (≥ 0.55), `ninja-build`, `pkg-config`, `gcc`, `g++` y `systemd`.

4.2 Instalación (compilación desde fuente)

Raspberry Pi OS no distribuye `mavlink-router` como paquete apt estable en todas sus versiones, así que la vía fiable es compilar desde el repositorio oficial. Para eso se usan `meson` y `ninja`: `meson` es la herramienta que lee la configuración del proyecto (qué ficheros hay que compilar, con qué opciones, qué dependencias necesita) y prepara todo lo necesario para construirlo; `ninja` es el programa que ejecuta esa compilación en sí, de forma rápida. En la práctica, para quien instala `mavlink-router` basta con saber que `meson setup` "prepara" la compilación y `ninja` la "ejecuta" — no hace falta más detalle que ese para seguir estos pasos:

```
sudo apt update
sudo apt install -y git meson ninja-build pkg-config gcc g++ systemd

cd ~
git clone https://github.com/mavlink-router/mavlink-router.git
cd mavlink-router
git submodule update --init --recursive

meson setup build .
ninja -C build
sudo ninja -C build install
```

```
mavlink-routerd --version    # confirma que el binario quedó instalado
```

Nota: Si meson del repositorio de Raspberry Pi OS es anterior a la 0.55 requerida, instalar una versión más reciente con "sudo pip3 install meson" (sin --user) antes de ejecutar meson setup.

4.3 Fichero de configuración: /etc/mavlink-router/main.conf

El fichero no se crea automáticamente en la instalación; hay que crearlo a mano en /etc/mavlink-router/main.conf. Un endpoint por proceso, todos en modo Normal (mavlink-router envía activamente a esa dirección:puerto, igual que ya hace Mission Planner en modo SITL — así el lado Python no cambia de patrón entre modos):

```
[General]
TcpServerPort = 0
ReportStats = false
MavlinkDialect = ardupilotmega

[UartEndpoint telem3]
Device = /dev/ttyAMA0
Baud = 57600

[UdpEndpoint receptor]
Mode = Normal
Address = 127.0.0.1
Port = 14550

[UdpEndpoint vuelo]
Mode = Normal
Address = 127.0.0.1
Port = 14551

# Opcional: un tercer endpoint para conectar QGroundControl/Mission
# Planner localmente a la Pi (p. ej. por SSH -L o VPN), sin usar TELEM1.
#[UdpEndpoint qgc]
#Mode = Normal
#Address = 127.0.0.1
#Port = 14552
```

- TcpServerPort=0 desactiva el servidor TCP por defecto (puerto 5760), que no se necesita aquí.
- Baud debe coincidir exactamente con el SERIALx_BAUD configurado en ArduPilot para TELEM3.
- Mode = Normal hace que mavlink-router envíe activamente cada mensaje a 127.0.0.1:puerto; receptor.py y vuelo.py simplemente escuchan ahí con udpin, igual que hoy escuchan el reenvío de Mission Planner. El modo alternativo, Mode = Server, obligaría a que el script emita un paquete primero para que mavlink-router "aprenda" su dirección — innecesario con este diseño.

mavlink-router: de un unico puerto serie a varios procesos

Cada endpoint UDP es bidireccional: heartbeats/telemetry bajan del FC a cada proceso, y comandos (arm, cambios de modo, request_data_stream) suben de cada proceso al FC.

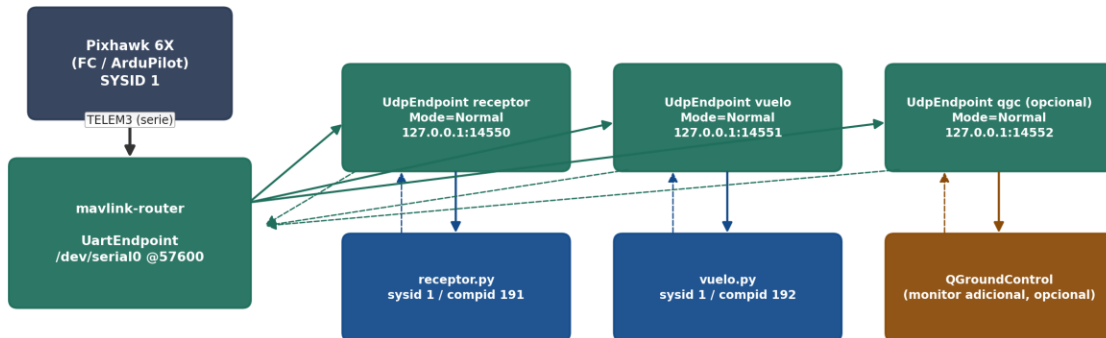


Figura 2. mavlink-router como único propietario del puerto serie, repartiendo el tráfico a un endpoint UDP por proceso.

4.4 Arrancar como servicio systemd

La instalación con "ninja install" registra por defecto la integración systemd. Para que mavlink-router arranque solo al iniciar la Raspberry Pi:

```
sudo systemctl daemon-reload
sudo systemctl enable mavlink-router.service
sudo systemctl start mavlink-router.service
sudo systemctl status mavlink-router.service

# Logs en vivo si algo falla:
journalctl -u mavlink-router.service -f
```

4.5 Verificación antes de conectar receptor.py y vuelo.py

Con el Pixhawk encendido y TELEM3 cableado, el servicio activo, y antes de tocar los scripts propios, conviene confirmar con una herramienta neutra que el enrutado funciona. Con mavproxy instalado (pip3 install mavproxy) o con un pequeño script pymavlink de prueba:

```
python3 -c "
from pymavlink import mavutil
m = mavutil.mavlink_connection('udpin:127.0.0.1:14550')
m.wait_heartbeat()
print('OK, sistema', m.target_system, 'componente', m.target_component)
"
```

- Repetir el mismo comando apuntando al puerto 14551 confirma que el segundo endpoint también recibe heartbeats simultáneamente — esa es la prueba de que el fan-out funciona antes de meter la lógica de receptor.py y vuelo.py.

5. receptor.py y vuelo.py: elegir el modo de conexión por parámetro

El objetivo es que ambos scripts sigan funcionando exactamente igual que hoy contra el SITL de Mission Planner, y que un único interruptor en el .env — sin tocar código — los conmute hacia el Pixhawk real a través de mavlink-router.

5.1 Diseño del .env unificado

Se añaden variables nuevas manteniendo compatibilidad con las que ya existían (MAVLINK_CONN y MAVLINK_CONN_VUELO se siguen respetando como alias del modo SITL si están definidas):

Variable	Significado y valor por defecto
MAVLINK_MODE	"sitr" (por defecto) o "real". Selecciona qué bloque de variables siguientes se usa.
MAVLINK_CONN_SITL_RECEPTOR	Conexión de receptor.py contra Mission Planner. Por defecto udpin:127.0.0.1:14550 (igual que hoy).
MAVLINK_CONN_SITL_VUELO	Conexión de vuelo.py contra Mission Planner. Por defecto udpin:0.0.0.0:14552 (igual que hoy).
MAVLINK_CONN_REAL_RECEPTOR	Conexión de receptor.py contra mavlink-router. Por defecto udpin:127.0.0.1:14550.
MAVLINK_CONN_REAL_VUELO	Conexión de vuelo.py contra mavlink-router. Por defecto udpin:127.0.0.1:14551.
MAVLINK_SYSID	SYSID que usan ambos procesos al emitir mensajes. Por defecto 1 (igual que el vehículo; ver sección 6).
MAVLINK_COMPID_RECEPTOR	COMPID propio de receptor.py. Por defecto 191.
MAVLINK_COMPID_VUELO	COMPID propio de vuelo.py. Por defecto 192.

Nota sobre los puertos: Los puertos 14550/14551 se reutilizan intencionadamente entre el modo SITL y el modo real para receptor.py: en cada momento solo uno de los dos procesos de origen (Mission Planner o mavlink-router) está activo alimentando ese puerto local, nunca los dos a la vez. vuelo.py mantiene 14552 como puerto SITL histórico (ya usado hoy para no chocar con receptor.py bajo Mission Planner) y pasa a 14551 en modo real, alineado con el endpoint de mavlink-router de la sección 4.3.

5.2 Cambios en receptor.py

En el bloque de configuración, sustituir la lectura de MAVLINK_CONN por esta lógica de selección de modo, y añadir sysid/compid propios:

```
# --- Selección de modo de conexión MAVLink ---
MAVLINK_MODE = os.getenv("MAVLINK_MODE", "sitr").strip().lower() # "sitr" | "real"

if MAVLINK_MODE == "real":
    MAVLINK_CONN = os.getenv(
        "MAVLINK_CONN_REAL_RECEPTOR", "udpin:127.0.0.1:14550")
else:
    # se mantiene compatibilidad con la variable MAVLINK_CONN original
```



```

MAVLINK_CONN = os.getenv(
    "MAVLINK_CONN_SITL_RECEPTOR",
    os.getenv("MAVLINK_CONN", "udpin:127.0.0.1:14550"))

# --- Identificación MAVLink propia de este proceso ---
MAVLINK_SYSID = int(os.getenv("MAVLINK_SYSID", 1))
MAVLINK_COMPID = int(os.getenv("MAVLINK_COMPID_RECEPTOR", 191))
# 191 = MAV_COMP_ID_ONBOARD_COMPUTER

```

Y en la conexión al autopiloto:

```

print(f"Conectando al autopiloto en {MAVLINK_CONN} "
      f"(modo={MAVLINK_MODE}, sysid={MAVLINK_SYSID}, compid={MAVLINK_COMPID}) ...")
master = mavutil.mavlink_connection(
    MAVLINK_CONN,
    source_system=MAVLINK_SYSID,
    source_component=MAVLINK_COMPID,
)
master.wait_heartbeat()

```

5.3 Cambios en vuelo.py

Misma lógica, aplicada a la variable existente MAVLINK_CONN_VUELO:

```

# --- Selección de modo de conexión MAVLink ---
MAVLINK_MODE = os.getenv("MAVLINK_MODE", "sitl").strip().lower() # "sitl" | "real"

if MAVLINK_MODE == "real":
    MAVLINK_CONN_VUELO = os.getenv(
        "MAVLINK_CONN_REAL_VUELO", "udpin:127.0.0.1:14551")
else:
    MAVLINK_CONN_VUELO = os.getenv(
        "MAVLINK_CONN_SITL_VUELO",
        os.getenv("MAVLINK_CONN_VUELO", "udpin:0.0.0.0:14552"))

MAVLINK_SYSID = int(os.getenv("MAVLINK_SYSID", 1))
MAVLINK_COMPID_VUELO = int(os.getenv("MAVLINK_COMPID_VUELO", 192))
# 192 = MAV_COMP_ID_ONBOARD_COMPUTER2

master = None
if not args.fake:
    print(f"Conectando al autopiloto en {MAVLINK_CONN_VUELO} "
          f"(modo={MAVLINK_MODE}, sysid={MAVLINK_SYSID}, "
          f"compid={MAVLINK_COMPID_VUELO}) ...")
    master = mavutil.mavlink_connection(
        MAVLINK_CONN_VUELO,
        source_system=MAVLINK_SYSID,
        source_component=MAVLINK_COMPID_VUELO,
    )
    master.wait_heartbeat()

```

Corrección necesaria: El docstring actual de vuelo.py menciona /dev/serial0 como ejemplo de ruta serie para el Pixhawk real. En esta placa concreta ese dispositivo no es el correcto (ver nota en 2.1) — el docstring debe actualizarse a /dev/ttyAMA0. Además, ahora que dos procesos comparten el autopiloto, ninguno de los dos debe apuntar directamente al dispositivo serie — ambos deben apuntar a su endpoint de mavlink-router.

5.4 Cómo alternar entre SITL y real

El cambio queda reducido a una línea en el .env, sin tocar código ni parámetros de línea de comandos:

```
# Para trabajar contra Mission Planner / SITL (por defecto):
MAVLINK_MODE=sitl

# Para trabajar contra el Pixhawk real, vía mavlink-router:
MAVLINK_MODE=real
```

6. SYSID / COMPID: por qué hay que añadirlo y cómo

6.1 Es necesario añadir identificación SYSID/COMPID propia a cada proceso

Cada mensaje MAVLink va etiquetado con un SYSID (identifica el sistema/vehículo de origen) y un COMPID (identifica el componente concreto dentro de ese sistema que lo envió). Es lo que permite que el autopiloto, Mission Planner, o cualquier herramienta de inspección MAVLink sepan de quién viene cada mensaje y a quién dirigir cada respuesta, y es la base para poder restringir en el futuro qué componente tiene permiso para hacer qué (por ejemplo, que solo receptor.py pueda armar el vehículo).

Ni receptor.py ni vuelo.py pasan hoy source_system ni source_component al crear la conexión (mavutil.mavlink_connection(MAVLINK_CONN) y mavutil.mavlink_connection(MAVLINK_CONN_VUELO), sin más argumentos), así que hay que añadirlo explícitamente. Al no indicarse, pymavlink aplica sus valores por defecto: source_system=255 y source_component=0 — los mismos que usan por convención las estaciones de tierra (Mission Planner incluido). Esto tiene consecuencias concretas mientras no se corrija:

- receptor.py y vuelo.py se identifican ante el autopiloto exactamente igual entre sí (255/0), y además de forma indistinguible de Mission Planner.
- En los logs del autopiloto (.bin/.tlog) y en cualquier inspector MAVLink no hay forma de saber qué mensaje vino de cuál de los tres.
- Si en algún momento dos de estos tres orígenes emiten mensajes con el mismo tipo pero contenido distinto casi simultáneamente (por ejemplo, dos REQUEST_DATA_STREAM con distinta frecuencia), no hay ninguna traza que permita diagnosticar el conflicto después del hecho.

Lo que hay que hacer es asignar a cada proceso un SYSID y un COMPID propios y pasarlos explícitamente a mavutil.mavlink_connection() mediante los parámetros source_system y source_component, siguiendo la convención MAVLink para computadoras de a bordo que se describe a continuación.

6.2 Convención propuesta

MAVLink reserva un rango de COMPID específico para computadoras de a bordo (compañeros de vuelo/companion computers): MAV_COMP_ID_ONBOARD_COMPUTER (191) a MAV_COMP_ID_ONBOARD_COMPUTER4 (194). La convención habitual para un companion computer es usar el mismo SYSID que el propio vehículo (por defecto 1 en ArduPilot, parámetro SYSID_THISMAV) y diferenciarse solo por COMPID:

Origen	SYSID	COMPID	Rol
Pixhawk 6X (autopiloto)	1	1 AUTOPILOT1	Vehículo — ya asignado por ArduPilot, no se toca.

Origen	SYSID	COMPID	Rol
receptor.py	1	191 ONBOARD_COMPUTER	Comandos: armar, desarmar, cambios de modo.
vuelo.py	1	192 ONBOARD_COMPUTER2	Solo lectura: telemetría de vuelo.
Mission Planner	255	0 GCS	Estación de tierra — sin cambios, valores por defecto de la herramienta.

6.3 Implementación

Ya recogida en la sección 5: en ambos scripts, las líneas MAVLINK_SYSID / MAVLINK_COMPID_RECEPTOR / MAVLINK_COMPID_VUELO junto con source_system= y source_component= al llamar a mavutil.mavlink_connection(). No hace falta ningún cambio adicional: pymavlink usa esos valores para todos los mensajes que el proceso emita a partir de ahí.

Posible incidencia a vigilar: Con algunas versiones de ArduPilot, ciertos comandos críticos (en particular cambios de modo y armado) sólo se aceptan si el componente de origen está reconocido como una estación de tierra válida. Hay reportes de usuarios de ArduPilot a los que el compid 191 (MAV_COMP_ID_ONBOARD_COMPUTER) les funcionó para telemetría pero no para comandos de armado/cambio de modo, teniendo que recurrir a compid 190 (MAV_COMP_ID_MISSIONPLANNER) para que el comando fuera aceptado. Esto es dependiente de la versión y configuración del autopiloto, no un comportamiento garantizado del estándar MAVLink. Si en las pruebas de la sección 7 el comando de armado desde receptor.py no es aceptado a pesar de heartbeats correctos, este es el primer punto a revisar — probar compid 190 para receptor.py como alternativa antes de sospechar de otra causa.

7. Procedimiento de prueba y validación

Antes de dar por buena la migración a modo real, seguir este orden — cada paso depende de que el anterior haya funcionado:

7.1 Banco, sin hélices

- Arrancar mavlink-router (systemctl status mavlink-router) y confirmar heartbeats en los puertos 14550 y 14551 con el comando de prueba de la sección 4.5.
- Poner MAVLINK_MODE=real en el .env y arrancar vuelo.py (modo solo lectura) primero. Confirmar en su log que llegan lat/lon/batería reales del Pixhawk, no ceros.
- Arrancar receptor.py y, sin enviar ningún comando MQTT todavía, confirmar el mensaje "Autopiloto conectado: sistema 1, componente 1" (el sysid/compid que reporta son los del Pixhawk, el target, no los propios).

7.2 Banco, sin hélices, con comando de armado

- Publicar un comando "arm" por MQTT contra receptor.py y confirmar en Mission Planner (que sigue viendo el vehículo por TELEM1/radio) que el estado pasa a ARMADO.
- Desarmar y repetir un par de veces para confirmar que el ciclo es estable.

Qué revisar si el armado no se acepta

- Confirmar por Mission Planner (Mavlink Inspector) que el mensaje COMMAND_LONG de armado llega con el sysid/compid esperado (1/191).
- Revisar ARMING_CHECK y los prerequisites habituales de armado (GPS fix, calibraciones) — un rechazo por estos motivos es indistinguible de un problema de compid si solo se mira el log de receptor.py.
- Si los prerequisites están bien y el comando sigue sin aceptarse, probar temporalmente MAVLINK_COMPID_RECEPTOR=190 como se indica en la nota de la sección 6.3.

7.3 Con hélices, dron sujeto/atado

- Repetir el armado con hélices montadas y el dron físicamente sujeto o en un banco de pruebas, confirmando que los motores responden y que el desarmado de emergencia (tanto por MQTT como por la emisora RC) corta inmediatamente.

7.4 Vuelo corto supervisado

- Primer vuelo en modo manual (STABILIZE/ALT_HOLD) para validar que el cambio de TELEM1/TELEM3 y la instalación de mavlink-router no han introducido ningún problema de vuelo en sí, con la emisora en la mano.
- Solo después, repetir con receptor.py/vuelo.py activos en modo real durante el vuelo, monitorizando Mission Planner por el radio de TELEM1 en todo momento y listo para tomar control manual.

8. Resumen: .env de ejemplo completo

El .env real que ya está en uso en la Raspberry Pi (drone-edge-companion) es compartido por varios procesos (sensor.py, vuelo.py, receptor.py, deteccin.py), y no todas las claves las lee cada uno. Antes de tocarlo, conviene tener claro qué usa cada proceso:

Bloque de claves	Claves	Quién las usa
Identificación del dron	DRON_ID	Todos los procesos
Broker MQTT	EC2_HOST, MQTT_PORT	Todos los procesos que publican/reciben por MQTT
Posición compartida	POS_FILE	vuelo.py (escribe), deteccin.py (lee)
Reenvío por lotes	LOTE	Procesos con buffer SQLite (sensor.py, vuelo.py)
MAVLink	MAVLINK_MODE, MAVLINK_CONN*, MAVLINK_SYSID, MAVLINK_COMPID_*	Únicamente receptor.py y vuelo.py — sensor.py y deteccin.py no las leen
Streaming de vídeo	STREAM_*	Únicamente deteccin.py (con --stream) — receptor.py y vuelo.py no las leen

Sobre ese .env real, los únicos cambios necesarios para lo descrito en este documento son las claves del bloque MAVLink: se mantienen MAVLINK_CONN y MAVLINK_CONN_VUELO tal cual ya están (son la conexión en modo SITL, y las usa el código como valor de reserva si no se define una variante específica), y se añaden

MAVLINK_MODE, las dos variables del modo real, y la identificación SYSID/COMPID de la sección 6. El resto del fichero no cambia:

```
# =====
# Configuración del nodo edge – varios procesos (sensor.py, vuelo.py,
# receptor.py, deteccion.py). No todas las claves se usan en todos.
# Sistema SAR basado en dron
# =====

# --- Identificación del dron (todos los procesos) ---
DRON_ID=dron-01

# --- Broker MQTT (procesos que publican/reciben por MQTT) ---
EC2_HOST=ec2-aws
MQTT_PORT=1883

# --- Fichero de posición (vuelo.py escribe, deteccion.py lee) ---
POS_FILE=./posicion_actual.json

# --- Reenvío por lotes (sensor.py, vuelo.py) ---
LOTE=50

# =====
# MAVLink – SOLO receptor.py y vuelo.py leen este bloque
# =====
# sitl -> Mission Planner / SITL (como hasta ahora)
# real -> mavlink-router / TELEM3 (sección 4)
MAVLINK_MODE=sitl

# --- Conexión en modo SITL: se mantienen los valores ya en uso ---
MAVLINK_CONN=udpin:0.0.0.0:14550
MAVLINK_CONN_VUELO=udpin:0.0.0.0:14552

# --- NUEVO: conexión en modo real, vía mavlink-router (sección 4.3) ---
MAVLINK_CONN_REAL_RECEPTOR=udpin:127.0.0.1:14550
MAVLINK_CONN_REAL_VUELO=udpin:127.0.0.1:14551

# --- NUEVO: identificación MAVLink de cada proceso (sección 6) ---
MAVLINK_SYSID=1
MAVLINK_COMPID_RECEPTOR=191
MAVLINK_COMPID_VUELO=192

# =====
# STREAMING DE VÍDEO EN DIRECTO (deteccion.py -> MediaMTX). Solo se usa
# con --stream. SIN CAMBIOS – se deja igual que en el .env actual.
# =====
STREAM_HOST=100.115.241.57
STREAM_USER=dron
STREAM_PASS=<mismo valor que ya tienes en tu .env – no se repite aquí>
STREAM_PATH=dron_live
STREAM_ANCHO=640
STREAM_ALTO=360
STREAM_FPS=12
```

Nota: MAVLINK_CONN y MAVLINK_CONN_VUELO ya usan udpin:0.0.0.0:14550/14552 en el .env real (escuchando en todas las interfaces), en vez de 127.0.0.1 como en algún ejemplo anterior de este documento

— es una diferencia menor y sin impacto: en SITL ambas formas funcionan igual de bien mientras Mission Planner reenvíe a ese puerto. No hace falta tocarlo.

8.1 Checklist final

- [] TELEM3 cableado a UART de la Raspberry Pi, SERIALx_PROTOCOL=2 y SERIALx_BAUD configurados en ArduPilot.
- [] UART habilitado en raspi-config, consola serie desactivada.
- [] mavlink-router compilado, instalado y con main.conf apuntando a /dev/ttyAMA0 (NO /dev/serial0 en esta placa) y a los puertos 14550/14551.
- [] Servicio mavlink-router.service activo y habilitado en el arranque.
- [] Prueba neutra de heartbeat en ambos puertos UDP (sección 4.5) superada.
- [] receptor.py y vuelo.py actualizados con selección de modo (MAVLINK_MODE) y sysid/compid propios.
- [] Comentario desactualizado sobre /dev/serial0 corregido a /dev/ttyAMA0 en vuelo.py.
- [] Pruebas de banco sin hélices, con hélices atado, y primer vuelo manual completadas en ese orden antes de operar con receptor.py/vuelo.py en vuelo real.
- [] test-serial.py / test-mavlink.py / test-arm-serial.py / test-estado.py guardados en la Raspberry Pi para futuras comprobaciones (Apéndice C).

Apéndice C — Scripts de prueba de la conexión serie TELEM3

Este apéndice documenta los cuatro scripts pequeños e independientes que se construyeron durante el diagnóstico de este proyecto para probar el enlace TELEM3 sin depender de mavlink-router ni de receptor.py/vuelo.py en marcha. Conviene guardarlos tal cual en la Raspberry Pi (por ejemplo en ~/drone-edge-companion/): son la forma más rápida de volver a comprobar el enlace en el futuro, o de diagnosticar un problema nuevo, sin tener que levantar todo el sistema.

Script	Para qué sirve
test-serial.py	Confirma que el UART de la Raspberry Pi funciona, sin ningún cable a la Pixhawk (loopback TX-RX en los propios pines 8 y 10).
test-mavlink.py	Confirma que llega un heartbeat real de la Pixhawk por TELEM3 — la primera prueba con el autopiloto de verdad conectado.
test-arm-serial.py	Prueba el ciclo completo de armado y desarmado (con confirmación de seguridad y opción de forzar el armado), por serie directo o a través de mavlink-router.
test-estado.py	Panel en vivo de batería, GPS y RC — incluye los mensajes reales de pre-arm del autopiloto — por serie directo o a través de mavlink-router.

C.1 test-serial.py — loopback en la propia Raspberry Pi

Es la prueba más básica y la que resolvió el caso real de este proyecto: comprueba que el UART de la Raspberry Pi funciona, sin ningún cable a la Pixhawk — se puentea directamente TX con RX en los propios pines 8 y 10 del header GPIO (sección 2.1), y el script envía un mensaje de texto por TX esperando recibirlo de

vuelta por RX. Si esto falla, el problema está en la propia Raspberry Pi (UART mal habilitado, dispositivo equivocado, o el puente físico mal puesto), no en el cable ni en la Pixhawk. Requiere el paquete pyserial (pip install pyserial dentro del entorno virtual del proyecto).

```
import serial
import time

DEVICE = '/dev/ttyAMA0'
BAUD = 57600
MENSAJE = b'TEST_LOOPBACK_OK\n'

print(f"Abriendo {DEVICE} a {BAUD} baudios...")
s = serial.Serial(DEVICE, BAUD, timeout=2)
s.reset_input_buffer()

print(f"Enviando: {MENSAJE}")
s.write(MENSAJE)
time.sleep(0.2)

respuesta = s.readline()
print(f"Recibido: {respuesta}")

if respuesta == MENSAJE:
    print("\n--> [OK] El puente TX-RX funciona correctamente.\n")
else:
    print(f"\n--> [FALLO] No ha llegado el mismo mensaje. Recibido: {respuesta!r}\n")

s.close()
```

C.2 test-mavlink.py — heartbeat real contra la Pixhawk

El mismo tipo de prueba que test-serial.py, pero ya con el cable de la Pixhawk conectado (no el jumper de loopback) y hablando MAVLink de verdad en vez de un mensaje de texto plano. Es la primera confirmación de que hay un piloto real respondiendo al otro lado del cable. Antes de ejecutarlo, hay que parar mavlink-router si está corriendo (sección 3.5), porque este script abre /dev/ttyAMA0 directamente y el puerto solo lo puede tener abierto un proceso a la vez.

```
from pymavlink import mavutil

DEVICE = '/dev/ttyAMA0'
BAUD = 57600

print(f"Conectando a {DEVICE} a {BAUD} baudios...")
m = mavutil.mavlink_connection(DEVICE, baud=BAUD)

print("Esperando heartbeat (puede tardar unos segundos)...")
m.wait_heartbeat()

print(f"\n--> OK: heartbeat recibido")
print(f"    Sistema: {m.target_system}")
print(f"    Componente: {m.target_component}")
```

C.3 test-arm-serial.py — ciclo completo de armado y desarmado

Va un paso más allá de test-mavlink.py: no solo confirma que hay conexión, sino que prueba el ciclo real de armar y desarmar el vehículo, con todo el detalle que hace falta para depurar un rechazo de armado sin tener que levantar receptor.py.

- Pide una confirmación explícita (escribir CONFIRMO) antes de tocar los motores — pensado para banco de pruebas con las hélices desmontadas, o con el dron sujeto/atado.
- Envía el ARM normal (respetando los pre-arm checks) y captura los STATUSTEXT reales del autopiloto si lo rechaza (p. ej. "PreArm: GPS: no fix", "Arm: Roll (RC1) is not neutral"), en vez de quedarse solo con un "no se ha podido armar" genérico.
- Si el ARM normal falla, pregunta explícitamente si se quiere FORZAR el armado saltándose esos chequeos (equivalente al botón "Force Arm" de Mission Planner), con su propia confirmación aparte — nunca se fuerza por accidente.
- Tras armar, espera 10 segundos (cuenta atrás en el log) y desarma, confirmando también el desarmado.

Por serie directo, o a través de mavlink-router

Admite el parámetro --conexion, con dos valores posibles:

- --conexion serial (por defecto): abre /dev/ttyAMA0 directamente. Si mavlink-router está corriendo como servicio, ya tiene el puerto serie abierto para él solo, así que este modo NO funcionará — hay que pararlo primero (sudo systemctl stop mavlink-router.service), o usar el modo router en su lugar.
- --conexion router: conecta por UDP contra el endpoint de mavlink-router (127.0.0.1:14550 por defecto) en vez de abrir el puerto serie. Es el modo a usar cuando mavlink-router ya está activo y no se quiere parar el servicio solo para hacer esta prueba.

Nota: El COMPID por defecto cambia según el modo (191 en serie, 193 por router) para no coincidir con el de receptor.py (191) si ambos hablan con mavlink-router al mismo tiempo — ver la convención de sysid/compid de la sección 6.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
=====
test-arm-disarm.py – Prueba de armado/desarmado por TELEM3 (serie)
Sistema SAR basado en dron – Raspberry Pi 5
=====

Conecta DIRECTAMENTE al Pixhawk por el puerto serie de TELEM3
(/dev/ttyAMA0 en esta Raspberry Pi – NO /dev/serial0, que en esta placa
apunta a un UART distinto que no está cableado a TELEM3), sin pasar por
mavlink-router ni por UDP. Es una prueba de un único proceso hablando
directo con el autopiloto, pensada para validar el ciclo completo de
armado antes de meter receptor.py/vuelo.py y mavlink-router de por medio.

Secuencia:
1. Conecta por serie y espera el heartbeat del autopiloto.
2. Pide confirmación explícita antes de armar (los motores pueden
   girar de verdad).
3. Arma los motores (respetando los pre-arm checks) y confirma que
   el autopiloto lo aceptó. Si lo rechaza, muestra el motivo
   (STATUSTEXT del autopiloto, p. ej. "PreArm: GPS fix") y pregunta
   si se quiere FORZAR el armado saltándose esos chequeos –
```


equivalente al botón "Force Arm" de Mission Planner, con su propia confirmación explícita aparte.

4. Espera 10 segundos, mostrando una cuenta atrás.

5. Desarma los motores y confirma que el autopiloto lo aceptó.

AVISO DE SEGURIDAD – leer antes de ejecutar:

- Las hélices deben estar DESMONTADAS, o el dron sujeto/atado en un banco de pruebas, tal y como se describe en el documento del proyecto (sección 7, "Procedimiento de prueba y validación").
- Si mavlink-router está corriendo, tiene el puerto serie abierto para él solo y este script no podrá conectar. Pararlo antes:

```
sudo systemctl stop mavlink-router.service
```

Y volver a arrancarlo después de la prueba:

```
sudo systemctl start mavlink-router.service
```

Uso:

```
python3 test-arm-disarm.py                # serie directo (por defecto)
python3 test-arm-disarm.py --conexion router  # vía mavlink-router (UDP)
python3 test-arm-disarm.py --device /dev/ttyAMA0 --baud 57600
python3 test-arm-disarm.py --segundos 15
python3 test-arm-disarm.py --sin-confirmar  # NO recomendado
"""

import sys
import time
import logging
import argparse
from datetime import datetime

from pymavlink import mavutil

# =====
# ARGUMENTOS DE LÍNEA DE COMANDOS
# =====
parser = argparse.ArgumentParser(
    description="Prueba de armado/desarmado del Pixhawk, por serie directo o por mavlink-router")
parser.add_argument("--conexion", choices=["serial", "router"], default="serial",
    help="'serial' (por defecto): abre el puerto serie directo (--device/--baud). "
    "'router': conecta contra mavlink-router por UDP (127.0.0.1:14550), "
    "sin tocar el puerto serie – usa esto si mavlink-router ya está "
    "corriendo.")
parser.add_argument("--device", default="/dev/ttyAMA0",
    help="Dispositivo serie, solo con --conexion serial (por defecto: "
    "/dev/ttyAMA0)")
parser.add_argument("--baud", type=int, default=57600,
    help="Baudios, solo con --conexion serial (por defecto: 57600, debe coincidir "
    "con SERIALx_BAUD)")
parser.add_argument("--puerto-router", default="udpin:127.0.0.1:14550",
    help="Endpoint UDP de mavlink-router, solo con --conexion router "
    "(por defecto: udpin:127.0.0.1:14550, el mismo que usa receptor.py)")
parser.add_argument("--sysid", type=int, default=1,
    help="SYSID propio de este proceso (por defecto: 1, igual que el vehículo)")
parser.add_argument("--compid", type=int, default=None,
    help="COMPID propio de este proceso. Por defecto: 191 con --conexion serial, "
    "193 con --conexion router (para no coincidir con receptor.py, que ya "
    "usa "
    "191 cuando habla contra mavlink-router al mismo tiempo).")
```

```

parser.add_argument("--segundos", type=int, default=10,
                    help="Segundos armado antes de desarmar (por defecto: 10)")
parser.add_argument("--sin-confirmar", action="store_true",
                    help="Salta la confirmación manual antes de armar (NO recomendado)")
parser.add_argument("--forzar-directo", action="store_true",
                    help="Si el ARM normal falla, fuerza directamente sin volver a preguntar "
                        "(sigue pidiendo la confirmación 'FORZAR' de seguridad)")
args = parser.parse_args()

if args.compilid is None:
    args.compilid = 193 if args.conexion == "router" else 191

# =====
# LOGGING – todo por pantalla, con hora exacta de cada paso
# =====
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s.%(msecs)03d [%(levelname)s] %(message)s",
    datefmt="%H:%M:%S",
    stream=sys.stdout,
)
log = logging.getLogger("test-arm-disarm")

def confirmar_seguridad():
    """Pide confirmación explícita antes de armar. No se puede saltar
    sin pasar --sin-confirmar a propósito.
    """
    log.warning("=" * 70)
    log.warning("AVISO DE SEGURIDAD: este script va a ARMAR los motores.")
    log.warning("Confirma que las hélices están DESMONTADAS, o que el dron")
    log.warning("está firmemente sujeto/atado en un banco de pruebas.")
    log.warning("=" * 70)
    respuesta = input("Escribe exactamente CONFIRMO para continuar: ").strip()
    if respuesta != "CONFIRMO":
        log.error("Confirmación no recibida ('%s'). Abortando sin tocar el autopiloto.",
            respuesta)
        sys.exit(1)
    log.info("Confirmación recibida. Continuando.")

def conectar(conexion, device, baud, puerto_router, sysid, compid):
    """Conecta al autopiloto, o bien por serie directo (TELEM3), o bien
    por UDP contra un endpoint de mavlink-router ya en marcha.
    """
    if conexion == "router":
        destino = puerto_router
        log.info("Conectando a mavlink-router en %s (sysid=%d, compid=%d) ...",
            destino, sysid, compid)
    else:
        destino = device
        log.info("Conectando a %s a %d baudios (sysid=%d, compid=%d) ...",
            destino, baud, sysid, compid)

    try:
        if conexion == "router":
            master = mavutil.mavlink_connection(
                destino,

```

```

        source_system=sysid,
        source_component=compid,
    )
else:
    master = mavutil.mavlink_connection(
        destino,
        baud=baud,
        source_system=sysid,
        source_component=compid,
    )
except Exception as e:
    log.error("No se ha podido abrir la conexión: %s", e)
    if conexion == "serial":
        log.error("¿No tendrás mavlink-router corriendo? Tiene el puerto serie "
            "abierto para él solo. Prueba: sudo systemctl stop mavlink-router.service "
            "- o, más simple, relanza este script con --conexion router.")
    else:
        log.error("¿Está mavlink-router realmente arrancado? Prueba: "
            "sudo systemctl status mavlink-router.service")
    sys.exit(1)

log.info("Conexión abierta. Esperando heartbeat del autopiloto (timeout 15s) ...")
hb = master.wait_heartbeat(timeout=15)
if hb is None:
    log.error("No ha llegado ningún heartbeat en 15 segundos.")
    if conexion == "serial":
        log.error("¿No tendrás mavlink-router corriendo y quedándose con el puerto "
            "serie? Prueba: sudo systemctl stop mavlink-router.service - o "
            "relanza este script con --conexion router.")
    else:
        log.error("¿Está mavlink-router corriendo, y con TELEM3 realmente recibiendo "
            "datos del Pixhawk? Revisa: journalctl -u mavlink-router.service -f")
    log.error("Si mavlink-router ya está descartado, revisa cableado (TX/RX/GND) y "
        "SERIALx_PROTOCOL/BAUD en ArduPilot.")
    sys.exit(1)

log.info("Heartbeat recibido. Autopiloto: sistema=%d, componente=%d",
    master.target_system, master.target_component)
log.info("Tipo de vehículo: %s | Autopiloto: %s",
    mavutil.mavlink.enums["MAV_TYPE"][hb.type].name,
    mavutil.mavlink.enums["MAV_AUTOPILOT"][hb.autopilot].name)
return master

MAGIC_FORZAR_ARM = 21196 # Valor especial que ArduPilot exige en param2 para saltarse
                        # los pre-arm checks (el mismo "Force Arm" de Mission Planner).

def _drenar_mensajes(master, tiempo_max=1):
    """Lee todos los mensajes que lleguen durante 'tiempo_max' segundos.

    Además de refrescar el estado de armado (vía HEARTBEAT), registra en el
    log cualquier STATUSTEXT del autopiloto – es el mecanismo por el que
    ArduPilot explica POR QUÉ rechaza un armado (p. ej. "PreArm: GPS fix").
    """
    limite = time.time() + tiempo_max
    while time.time() < limite:
        msg = master.recv_match(blocking=True, timeout=max(0.0, limite - time.time()))

```

```

        if msg is None:
            break
        if msg.get_type() == "STATUSTEXT":
            log.warning(" [STATUSTEXT autopiloto] %s", msg.text)

def esperar_modoy_estado(master, timeout=5):
    """Registra el modo de vuelo actual y el estado de armado antes de tocar nada."""
    master.recv_match(type="HEARTBEAT", blocking=True, timeout=timeout)
    modo = master.flightmode or "DESCONOCIDO"
    armado = master.motors_armed()
    log.info("Estado actual -> modo: %s | armado: %s", modo, armado)
    if armado:
        log.warning("El vehículo YA estaba armado antes de empezar la prueba.")
    return modo, armado

def armar(master, timeout=10):
    """Envía el comando de armado normal (respeta los pre-arm checks) y
    espera confirmación del autopiloto, mostrando cualquier STATUSTEXT
    que explique un posible rechazo.
    """
    log.info("Enviando comando ARM (respetando pre-arm checks) ...")
    t0 = time.time()
    master.arducopter_arm()

    log.info("Esperando confirmación de armado (timeout %ds) ...", timeout)
    limite = time.time() + timeout
    while time.time() < limite and not master.motors_armed():
        _drenar_mensajes(master, tiempo_max=1)

    if not master.motors_armed():
        log.error("El autopiloto NO ha aceptado el armado en %d segundos.", timeout)
        log.error("Si arriba no ha aparecido ningún [STATUSTEXT autopiloto] con el "
            "motivo, revisa igualmente ARMING_CHECK y los prerrequisitos "
            "habituales (GPS fix, calibraciones, failsafes) en Mission Planner.")
        return False

    log.info("ARMADO confirmado por el autopiloto (%.2fs).", time.time() - t0)
    return True

def armar_forzado(master, timeout=10):
    """Envía el comando de armado FORZADO, saltándose los pre-arm checks
    (equivalente al botón 'Force Arm' de Mission Planner). Usa
    MAV_CMD_COMPONENT_ARM_DISARM con el valor mágico 21196 en param2.
    """
    log.warning("Enviando comando ARM FORZADO (saltando pre-arm checks) ...")
    t0 = time.time()
    master.mav.command_long_send(
        master.target_system,
        master.target_component,
        mavutil.mavlink.MAV_CMD_COMPONENT_ARM_DISARM,
        0,          # confirmation
        1,          # param1: 1 = armar
        MAGIC_FORZAR_ARM, # param2: valor mágico para forzar
        0, 0, 0, 0, 0,
    )

```

```

log.info("Esperando confirmación de armado forzado (timeout %ds) ...", timeout)
limite = time.time() + timeout
while time.time() < limite and not master.motors_armed():
    _drenar_mensajes(master, tiempo_max=1)

if not master.motors_armed():
    log.error("El autopiloto NO ha aceptado el armado NI SIQUIERA FORZADO en "
              "%d segundos. Esto ya no es un simple pre-arm check – revisa la "
              "conexión, el failsafe de radio, o si hay algún bloqueo por "
              "software (safety switch físico, si tu Pixhawk lo tiene).", timeout)
    return False

log.info("ARMADO FORZADO confirmado por el autopiloto (%.2fs).", time.time() - t0)
return True

def desarmar(master, timeout=10):
    """Envía el comando de desarmado y espera confirmación del autopiloto."""
    log.info("Enviando comando DISARM ...")
    t0 = time.time()
    master.arducopter_disarm()

    limite = time.time() + timeout
    while time.time() < limite and master.motors_armed():
        _drenar_mensajes(master, tiempo_max=1)

    if master.motors_armed():
        log.error("El autopiloto sigue ARMADO tras %d segundos intentando desarmar.", timeout)
        log.error("¡Atención! Desarma manualmente desde la emisora RC o Mission "
                  "Planner de inmediato.")
        return False

    log.info("DESARMADO confirmado por el autopiloto (%.2fs).", time.time() - t0)
    return True

def cuenta_atras(segundos):
    """Muestra en el log una cuenta atrás mientras el vehículo está armado."""
    log.info("Vehículo armado. Esperando %d segundos antes de desarmar ...", segundos)
    for restante in range(segundos, 0, -1):
        log.info(" ... %d s", restante)
        time.sleep(1)

def confirmar_forzar():
    """Pide una segunda confirmación, más explícita, antes de forzar el
    armado saltándose los pre-arm checks. Nunca se salta con --sin-confirmar
    (ese flag solo afecta a la primera confirmación).
    """
    log.warning("=" * 70)
    log.warning("El comando ARM normal no ha funcionado porque no cumple algún")
    log.warning("chequeo de pre-armado (ARMING_CHECK). ¿Quieres FORZAR el ARM,")
    log.warning("saltándote esos chequeos? Es lo mismo que 'Force Arm' en Mission")
    log.warning("Planner: los motores pueden arrancar aunque algo no esté bien")
    log.warning("(por ejemplo, sin fix GPS o sin calibrar).")
    log.warning("=" * 70)
    respuesta = input("Escribe exactamente FORZAR para continuar, o cualquier otra cosa para
abortar: ").strip()

```

```

return respuesta == "FORZAR"

def main():
    log.info("=" * 70)
    log.info("test-arm-disarm.py - inicio %s", datetime.now().isoformat(timespec="seconds"))
    log.info("=" * 70)

    master = conectar(args.conexion, args.device, args.baud, args.puerto_router,
                      args.sysid, args.compид)
    esperar_modo_y_estado(master)

    if not args.sin_confirmar:
        confirmar_seguridad()
    else:
        log.warning("Confirmación de seguridad SALTADA (--sin-confirmar). Continuando bajo tu
responsabilidad.")

    armado_ok = armar(master)
    if not armado_ok:
        if args.forzar_directo:
            proceder_forzado = True
        else:
            proceder_forzado = confirmar_forzar()

        if not proceder_forzado:
            log.error("Prueba ABORTADA: no se ha podido armar (normal) y no se ha "
                    "confirmado el armado forzado. No se intenta desarmar (no "
                    "hacía falta, nunca llegó a armar).")
            sys.exit(2)

        armado_ok = armar_forzado(master)
        if not armado_ok:
            log.error("Prueba ABORTADA: tampoco se ha podido armar de forma forzada. "
                    "No se intenta desarmar (no hacía falta, nunca llegó a armar).")
            sys.exit(2)

    try:
        cuenta_atras(args.segundos)
    except KeyboardInterrupt:
        log.warning("Interrumpido por el usuario (Ctrl+C). Desarmando inmediatamente.")

    desarmado_ok = desarmar(master)

    log.info("=" * 70)
    if armado_ok and desarmado_ok:
        log.info("PRUEBA COMPLETADA CON ÉXITO: armado y desarmado confirmados por el
autopiloto.")
    else:
        log.error("PRUEBA COMPLETADA CON ERRORES. Revisa los mensajes anteriores.")
    log.info("=" * 70)

if __name__ == "__main__":
    main()

```

C.4 test-estado.py — panel en vivo de batería, GPS y RC

Un panel que se refresca cada segundo en la terminal, útil para vigilar el estado del vehículo mientras se ajustan cosas (calibración de radio, posicionamiento para fix GPS, etc.) sin tener que interpretar mensajes MAVLink sueltos a mano.

- BATERÍA: voltaje, corriente, porcentaje restante, con aviso si baja del 20%.
- GPS: tipo de fix, satélites visibles, HDOP, posición, con aviso si todavía no hay fix 3D.
- RC: si el receptor está presente y sano, valores en crudo de los 8 primeros canales, RSSI (con la aclaración de que muchos receptores PPM/SBUS no lo reportan, sin que eso sea un fallo), y un aviso heurístico si el throttle no parece estar al mínimo.
- PRE-ARM: los mensajes STATUSTEXT reales del autopiloto explicando qué falla para poder armar. El script pide activamente este re-chequeo cada pocos segundos (MAV_CMD_RUN_PREARM_CHECKS) en vez de esperar al ciclo automático de ~30s de ArduPilot — es la única fuente fiable de este dato, ya que ArduPilot nunca ha implementado el bit de "listo para armar" del estándar MAVLink en SYS_STATUS (issue conocido y abierto en su GitHub, #13534).

Admite el mismo parámetro --conexion serial|router que test-arm-serial.py, con la misma lógica: usar router si mavlink-router ya está corriendo y no se quiere parar el servicio.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
=====
test-estado.py – Panel en vivo: batería, GPS y RC del Pixhawk
Sistema SAR basado en dron – Raspberry Pi 5
=====

Conecta al autopiloto (por serie directo o por mavlink-router, igual
que test-arm-disarm.py) y muestra un panel que se refresca cada segundo
con:

- BATERÍA: voltaje, corriente, porcentaje restante.
- GPS: tipo de fix, satélites visibles, HDOP, posición.
- RC: si el receptor está presente y sano, valores en crudo de los
  canales, y un aviso heurístico de si el throttle parece estar en
  una posición que bloquearía el armado (no al mínimo).
- PRE-ARM: los mensajes STATUSTEXT reales que envía ArduPilot
  explicando qué falla para poder armar (p. ej. "PreArm: GPS: no
  fix"). El script pide activamente estos mensajes cada pocos
  segundos (MAV_CMD_RUN_PREARM_CHECKS), porque ArduPilot NO expone
  un booleano fiable de "listo para armar" en SYS_STATUS – es un
  hueco conocido y nunca implementado del firmware (ver issue
  ArduPilot/ardupilot #13534). Esta es la única fuente real de ese
  dato por MAVLink.

No modifica nada del vehículo salvo pedir el re-chequeo de pre-arm
(MAV_CMD_RUN_PREARM_CHECKS), que no arma ni cambia ningún parámetro: es
de solo lectura para todo lo demás.

AVISO sobre el chequeo de RC: es una comprobación HEURÍSTICA basada en
el valor en crudo del canal de throttle (por defecto, canal 3). No
sustituye a los pre-arm checks reales de ArduPilot – para ver el motivo
EXACTO de un rechazo de armado, usa test-arm-disarm.py, que captura los
mensajes STATUSTEXT del propio autopiloto.

Uso:
```

```

python3 test-estado.py                # serie directo (por defecto)
python3 test-estado.py --conexion router # vía mavlink-router (UDP)
python3 test-estado.py --canal-throttle 3 # cambiar el canal si RCMAP_THROTTLE no es el 3
(Ctrl+C para salir)
"""

import sys
import time
import shutil
import logging
import argparse

from pymavlink import mavutil

# =====
# ARGUMENTOS DE LÍNEA DE COMANDOS
# =====
parser = argparse.ArgumentParser(
    description="Panel en vivo de batería, GPS y RC del Pixhawk, por serie o por mavlink-router")
parser.add_argument("--conexion", choices=["serial", "router"], default="serial",
    help="'serial' (por defecto): puerto serie directo (--device/--baud). "
    "'router': UDP contra mavlink-router (127.0.0.1:14550), sin tocar "
    "el puerto serie – usa esto si mavlink-router ya está corriendo.")
parser.add_argument("--device", default="/dev/ttyAMA0",
    help="Dispositivo serie, solo con --conexion serial (por defecto: "
    "/dev/ttyAMA0)")
parser.add_argument("--baud", type=int, default=57600,
    help="Baudios, solo con --conexion serial (por defecto: 57600)")
parser.add_argument("--puerto-router", default="udpin:127.0.0.1:14550",
    help="Endpoint UDP de mavlink-router, solo con --conexion router "
    "(por defecto: udpin:127.0.0.1:14550)")
parser.add_argument("--sysid", type=int, default=1,
    help="SYSID propio de este proceso (por defecto: 1, igual que el vehículo)")
parser.add_argument("--compid", type=int, default=None,
    help="COMPID propio de este proceso. Por defecto: 191 con --conexion serial, "
    "194 con --conexion router (para no coincidir con receptor.py=191 ni "
    "vuelo.py=192 si están hablando con mavlink-router a la vez).")
parser.add_argument("--canal-throttle", type=int, default=3,
    help="Número de canal RC que corresponde al throttle (por defecto: 3, "
    "el estándar de ArduPilot; ajústalo si tu RCMAP_THROTTLE es distinto)")
parser.add_argument("--frecuencia", type=float, default=1.0,
    help="Refresco del panel en segundos (por defecto: 1.0)")
args = parser.parse_args()

if args.compid is None:
    args.compid = 194 if args.conexion == "router" else 191

# =====
# LOGGING (solo para errores de arranque; el panel usa print directo)
# =====
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s.%(msecs)03d [%(levelname)s] %(message)s",
    datefmt="%H:%M:%S",
    stream=sys.stderr,
)

```



```

log = logging.getLogger("test-estado")

# =====
# CONEXIÓN
# =====
def conectar():
    if args.conexion == "router":
        destino = args.puerto_router
        log.info("Conectando a mavlink-router en %s (sysid=%d, compid=%d) ...",
                destino, args.sysid, args.compid)
        kwargs = {}
    else:
        destino = args.device
        log.info("Conectando a %s a %d baudios (sysid=%d, compid=%d) ...",
                destino, args.baud, args.sysid, args.compid)
        kwargs = {"baud": args.baud}

    try:
        master = mavutil.mavlink_connection(
            destino, source_system=args.sysid, source_component=args.compid, **kwargs)
    except Exception as e:
        log.error("No se ha podido abrir la conexión: %s", e)
        if args.conexion == "serial":
            log.error("¿No tendrás mavlink-router corriendo? Prueba --conexion router, "
                    "o sudo systemctl stop mavlink-router.service")
        else:
            log.error("¿Está mavlink-router realmente arrancado? "
                    "sudo systemctl status mavlink-router.service")
        sys.exit(1)

    log.info("Conexión abierta. Esperando heartbeat (timeout 15s) ...")
    hb = master.wait_heartbeat(timeout=15)
    if hb is None:
        log.error("No ha llegado ningún heartbeat en 15 segundos.")
        if args.conexion == "serial":
            log.error("¿No tendrás mavlink-router corriendo y quedándose con el puerto? "
                    "Prueba --conexion router, o para mavlink-router primero.")
        else:
            log.error("¿mavlink-router está corriendo y TELEM3 recibe datos del Pixhawk? "
                    "Revisa: journalctl -u mavlink-router.service -f")
        sys.exit(1)

    log.info("Heartbeat recibido: sistema=%d, componente=%d",
            master.target_system, master.target_component)

    # Pide explícitamente todos los streams de datos a 4 Hz – sin esto,
    # el autopiloto puede no enviar por su cuenta todo lo que necesitamos.
    master.mav.request_data_stream_send(
        master.target_system, master.target_component,
        mavutil.mavlink.MAV_DATA_STREAM_ALL, 4, 1)

    return master

def pedir_prearm_checks(master):
    """Pide al autopiloto que vuelva a evaluar y enviar por STATUSTEXT el
    motivo de cualquier fallo de pre-arm, sin esperar a su ciclo automático
    de ~30s. ArduPilot no expone un booleano fiable de "listo para armar"

```

```

en SYS_STATUS (bug conocido, nunca implementado) – los STATUSTEXT
"PreArm: ..." son la única fuente real de este dato.
"""

master.mav.command_long_send(
    master.target_system, master.target_component,
    mavutil.mavlink.MAV_CMD_RUN_PREARM_CHECKS,
    0, 0, 0, 0, 0, 0, 0, 0,
)

# =====
# INTERPRETACIÓN DE MENSAJES
# =====
FIX_TYPE_NOMBRES = {
    0: "SIN GPS", 1: "SIN FIX", 2: "FIX 2D", 3: "FIX 3D",
    4: "DGPS", 5: "RTK FLOAT", 6: "RTK FIJO", 7: "ESTÁTICO", 8: "PPP",
}

SENSOR_RC_RECEIVER = mavutil.mavlink.MAV_SYS_STATUS_SENSOR_RC_RECEIVER

def resumen_bateria(msgs):
    sys_status = msgs.get("SYS_STATUS")
    bat = msgs.get("BATTERY_STATUS")

    if sys_status is None and bat is None:
        return [" (todavía sin datos de batería)"]

    lineas = []
    if sys_status is not None:
        v = sys_status.voltage_battery
        i = sys_status.current_battery
        pct = sys_status.battery_remaining
        v_txt = f"{v/1000:.2f} V" if v != 65535 else "N/D"
        i_txt = f"{i/100:.1f} A" if i != -1 else "N/D"
        pct_txt = f"{pct} %" if pct != -1 else "N/D"
        lineas.append(f" Voltaje: {v_txt} Corriente: {i_txt} Restante: {pct_txt}")
        if pct != -1 and pct < 20:
            lineas.append(" ⚠ Batería por debajo del 20% – no volar.")
    return lineas

def resumen_gps(msgs):
    gps = msgs.get("GPS_RAW_INT")
    if gps is None:
        return [" (todavía sin datos de GPS)"]

    fix_txt = FIX_TYPE_NOMBRES.get(gps.fix_type, f"desconocido ({gps.fix_type})")
    sats = gps.satellites_visible if gps.satellites_visible != 255 else "N/D"
    hdop = gps.eph / 100.0 if gps.eph != 65535 else None
    lat = gps.lat / 1e7
    lon = gps.lon / 1e7

    lineas = [f" Fix: {fix_txt} Satélites: {sats}" +
              (f" HDOP: {hdop:.2f}" if hdop is not None else "")]
    if gps.fix_type >= 3:
        lineas.append(f" Posición: {lat:.7f}, {lon:.7f}")
    else:

```

```

        lineas.append("  ⚠ Sin fix 3D todavía – ArduPilot normalmente no arma sin GPS "
                      "(salvo que ARMING_CHECK lo excluya).")
    return lineas

def resumen_rc(msgs, canal_throttle):
    sys_status = msgs.get("SYS_STATUS")
    rc = msgs.get("RC_CHANNELS")

    lineas = []

    if sys_status is not None:
        presente = bool(sys_status.onboard_control_sensors_present & SENSOR_RC_RECEIVER)
        sano = bool(sys_status.onboard_control_sensors_health & SENSOR_RC_RECEIVER)
        if not presente:
            lineas.append("  ⚠ El autopiloto no reporta ningún receptor RC presente.")
        elif not sano:
            lineas.append("  ⚠ Receptor RC presente pero marcado como NO SANO "
                          "(posible failsafe o pérdida de señal).")
        else:
            lineas.append("  Receptor RC: presente y sano.")

    if rc is None:
        lineas.append("  (todavía sin datos de canales RC)")
        return lineas

    canales = [rc.chan1_raw, rc.chan2_raw, rc.chan3_raw, rc.chan4_raw,
               rc.chan5_raw, rc.chan6_raw, rc.chan7_raw, rc.chan8_raw]
    etiquetas = ["ROLL", "PITCH", "THR", "YAW", "AUX1", "AUX2", "AUX3", "AUX4"]
    fila = " " + " ".join(f"{et}={val}" for et, val in zip(etiquetas, canales))
    lineas.append(fila)

    rssi = rc.rssi
    if rssi == 255:
        lineas.append("  RSSI: no reportado por este receptor/protocolo (normal en muchos "
                      "receptores PPM/SBUS sin RSSI_TYPE configurado – no es un problema si "
                      "los valores de canal de arriba cambian con la emisora).")
    elif rssi == 0:
        lineas.append("  ⚠ RSSI en 0 – si además los canales de arriba no se mueven al mover "
                      "la emisora, sí podría ser pérdida de señal real. Si los canales cambian "
                      "con normalidad, probablemente sea el mismo caso de \"no reportado\".")
    else:
        lineas.append(f"  RSSI: {rssi}")

    idx = canal_throttle - 1
    if 0 <= idx < len(canales):
        val_thr = canales[idx]
        if val_thr == 0:
            lineas.append(f"  ⚠ Canal {canal_throttle} (throttle) en 0 – no hay señal RC "
                          "real en ese canal, revisa --canal-throttle.")
        elif val_thr > 1200:
            lineas.append(f"  ⚠ Throttle (canal {canal_throttle}) = {val_thr}, no parece "
                          "estar al mínimo – esto suele BLOQUEAR el armado en ArduPilot. "
                          "Baja el stick de gas del todo.")
        else:
            lineas.append(f"  Throttle (canal {canal_throttle}) = {val_thr} – parece al "
                          "mínimo, correcto para armar.")
    else:
        lineas.append("  (canal_throttle fuera de rango)")
    return lineas

```

```

return lineas

def resumen_general(master, msgs):
    modo = master.flightmode or "DESCONOCIDO"
    armado = master.motors_armed()
    lineas = [f" Modo: {modo}   Armado: {'SÍ' if armado else 'no'}"]
    return lineas

# =====
# BUCLE PRINCIPAL – panel que se refresca
# =====
def limpiar_pantalla():
    print("\033[H\033[J", end="")

def main():
    master = conectar()
    print() # deja el log de conexión visible antes del primer refresco

    prearm_msgs = [] # últimos STATUSTEXT relacionados con pre-arm
    ultima_peticion = 0.0
    INTERVALO_PETICION = 6.0 # cada cuántos segundos forzar un re-chequeo

    try:
        while True:
            ahora = time.time()
            if ahora - ultima_peticion >= INTERVALO_PETICION:
                pedir_pream_checks(master)
                ultima_peticion = ahora

            # Drena todo lo recibido en esta vuelta, sin bloquear, y de paso
            # captura cualquier STATUSTEXT que llegue (no queda en .messages,
            # solo se ve el último de cada tipo, así que hay que cogerlo aquí).
            while True:
                msg = master.recv_match(blocking=False)
                if msg is None:
                    break
                if msg.get_type() == "STATUSTEXT":
                    texto = msg.text.strip()
                    if texto and (not prearm_msgs or prearm_msgs[-1] != texto):
                        prearm_msgs.append(texto)
                        prearm_msgs[:] = prearm_msgs[-5:] # solo los 5 últimos

            msgs = master.messages

            ancho = shutil.get_terminal_size((80, 24)).columns
            limpiar_pantalla()
            print("=" * ancho)
            print(" ESTADO DEL PIXHAWK – Ctrl+C para salir".ljust(ancho))
            print("=" * ancho)

            print("\nGENERAL")
            for l in resumen_general(master, msgs):
                print(l)

            print("\nBATERÍA")

```

```

    for l in resumen_bateria(msgs):
        print(l)

    print("\nGPS")
    for l in resumen_gps(msgs):
        print(l)

    print("\nRC")
    for l in resumen_rc(msgs, args.canal_throttle):
        print(l)

    print("\nPRE-ARM (mensajes del autopiloto)")
    if prearm_msgs:
        for texto in prearm_msgs:
            print(f" {texto}")
    else:
        print(" (sin mensajes todavía – si no aparece nada en unos segundos, "
              "puede que ya esté listo para armar, o que no reciba STATUSTEXT)")

    print("\n" + "=" * ancho)
    time.sleep(args.frecuencia)

except KeyboardInterrupt:
    print("\nDetenido por el usuario.")

if __name__ == "__main__":
    main()

```